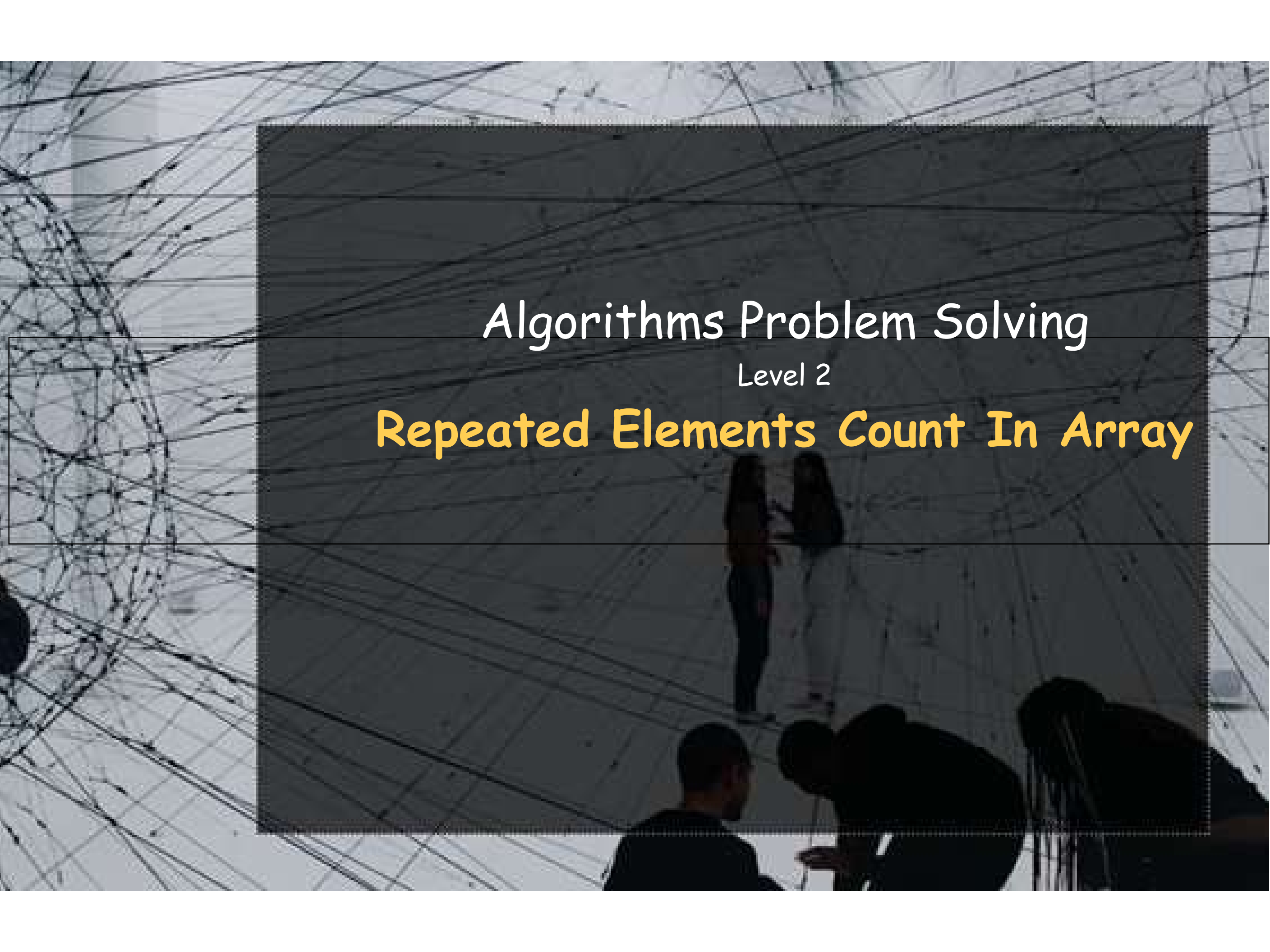


Algorithms Problem Solving Level 2

Repeated Elements Count In Array



Algorithms Problem Solving

Level 2

Repeated Elements Count In Array

Definition

- ◆ A program that reads a number of elements, stores them in an array, prints the array, and counts how many times a specific number appears in it.
- ◆ The program uses an array and a loop to search for repeated elements.

Explanation

- ◆ ``ReadNumber()`` reads a positive number and uses a ``do-while`` loop to make sure the value is greater than ``0``.
- ◆ ``ReadArrayElements()`` reads the array elements one by one and stores them inside ``arr``.
- ◆ ``PrintArrayElements()`` loops through the array and prints all its elements.
- ◆ ``CountFrequency()`` searches through the array and compares each element with ``numberToCheck``.
- ◆ If ``arr[i] == numberToCheck``, the ``count`` variable is increased by ``1``.
- ◆ After checking all elements, ``CountFrequency()`` returns the total number of occurrences.
- ◆ The array has a maximum capacity of ``100`` elements because it is declared as ``int arr[100]``.

Steps in main()

1. Read the array length from the user.
2. Create an array with a maximum capacity of `100` elements.
3. Read each element and store it in the array.
4. Ask the user for the number to check.
5. Print all elements of the array.
6. Call `CountFrequency()`.
7. Compare every array element with `numberToCheck`.
8. Increase `count` whenever a match is found.
9. Return the final count.
10. Print how many times the number appears in the array.

💡 Example

◆ Input:

Enter array length: 5

Element[1]: 1

Element[2]: 1

Element[3]: 1

Element[4]: 2

Element[5]: 3

Enter the number you want to check: 1

The array is:

1 1 1 2 3

◆ The number `1` appears 3 times.

➡ Result

The program prints:

Original array: 1 1 1 2 3

1 is repeated 3 time(s).

`CountFrequency()` checks all 5 elements and finds three matches.

📌 Note

- ◆ `arr[i]` represents the element at index `i`.
- ◆ Array indexes start from `0`, so:
 - `arr[0]` → first element
 - `arr[1]` → second element
 - `arr[2]` → third element
- ◆ The expression `i + 1` is used when printing the element number so the user sees `Element[1]`, `Element[2]`, etc.
- ◆ `count` starts at `0` because no matching elements have been found yet.
- ◆ Every time `arr[i] == numberToCheck`, `count++` increases the frequency by one.

⊘ Warning

- ◆ `ReadNumber()` only checks that the array length is greater than `0`; it does not check that the length is `100` or less.
- ◆ If the user enters a length greater than `100`, the program can access memory outside the array.
- ◆ The code includes `<cmath>` and `<corecrt_math_defines.h>`, but they are not needed for this program.

Comparison

- ▣ `ReadArrayElements()` → stores values inside the array.
- ▣ `PrintArrayElements()` → displays the values stored in the array.
- ▣ `CountFrequency()` → searches the array and counts matching values.
- ▣ `arr[i] == numberToCheck` → checks whether the current element matches the number we are looking for.
- ▣ `count++` → increases the number of matches found.

✦ Summary

- ✓ The program stores elements inside an array.
- ✓ `ReadArrayElements()` fills the array.
- ✓ `PrintArrayElements()` prints the array.
- ✓ `CountFrequency()` counts how many times a number appears.
- ✓ `arr[i] == numberToCheck` checks for a match.
- ✓ `count++` increases the frequency when a match is found.
- ✓ The array can store up to `100` elements with the current declaration.

Thank you 😊